

B.M.S. COLLEGE OF ENGINEERING BENGALURU
Autonomous Institute, Affiliated to VTU



OOMD Mini Project Report

Smart Curriculum Activity & Attendance App

Submitted in partial fulfillment for the award of degree of

Bachelor of Engineering
in
Computer Science and Engineering

Submitted by:

Siddharth Arya

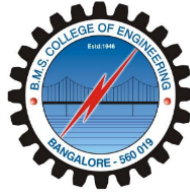
1BM23CS328

Shashank Shantharam Nayak

1BM23CS313

Department of Computer Science and Engineering
B.M.S. College of Engineering
Bull Temple Road, Basavanagudi, Bangalore 560 019
Aug 2025 - Dec 2025

B.M.S. COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

We, Siddharth Arya (1BM23CS328), Shashank Shantharam Nayak (1BM23CS313) students of 5th Semester, B.E, Department of Computer Science and Engineering, BMS College of Engineering, Bangalore, hereby declare that, this OOMD Mini Project entitled "Smart Curriculum Activity & Attendance App" has been carried out in Department of CSE, B.M.S. College of Engineering, Bangalore during the academic semester Aug 2025 to Dec 2025. I also declare that to the best of our knowledge and belief, the OOMD mini Project report is not from part of any other report by any other students.

Signature of the Candidate

Siddharth Arya (1BM23CS328)

Shashank Shantharam Nayak (1BM23CS313)

B.M.S. COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING



CERTIFICATE

This is to certify that the OOMD Mini Project titled “**Smart Curriculum Activity & Attendance App**” has been carried out by Siddharth Arya (1BM23CS328), Shashank Shantharam Nayak (1BM23CS313) during the academic year 2024-2025.

Signature of the Faculty in Charge

Table of Contents

Sl No	Title	Page no.
1	Ch 1: Problem statement	5
2	Ch 2: Software Requirement Specification	7
3	Ch 3: Class Diagram	13
4	Ch 4: State Diagram	17
5	Ch 5: Interaction diagram	18
6	Ch 6: UI Design with Screenshots	27

Chapter 1: Problem Statement

Problem Description

Many educational institutions still depend on manual attendance systems, which are time-consuming and error-prone. Teachers spend a significant portion of class time marking attendance, reducing valuable instructional hours. Additionally, students often waste free periods with unproductive activities due to a lack of structured guidance. This leads to poor time management and reduced alignment with long-term academic or career goals. There is also a gap in personalized learning support during idle classroom hours. Institutions currently lack tools that integrate daily schedules with individual student planning and automated tracking.

Impact / Why this problem needs to be solved

This issue impacts both administrative efficiency and student development. Automating attendance saves teachers' time and ensures more accurate records. Additionally, providing students with structured personal development activities during free time helps improve productivity, goal clarity, and learning outcomes. Institutions can also gain better insight into student behavior and engagement, allowing for more targeted support.

Expected Outcomes

- Automatically marks student attendance based on the daily timetable using QR code, Bluetooth/Wi-Fi proximity, or face recognition.
- Displays real-time attendance on a classroom screen.
- Suggests personalized academic tasks during free periods based on the student's interests, strengths, and career goals.
- Generates a daily routine combining class schedule, free time, and long-term personal goals.

The app will require minimal infrastructure and be usable by both students and staff with basic training.

Relevant Stakeholders / Beneficiaries

- Students
- Teachers
- College administrators
- Career counselors
- Education departments

Supporting Data

- Surveys and reports on classroom time usage, student productivity, and NEP 2020 recommendations emphasizing personalized and experiential learning.

Chapter 2: Software Requirement Specification

1. Introduction

1.1 Purpose of this Document

This document outlines the software requirements for the Smart Curriculum Activity & Attendance App, which aims to automate attendance management and optimize students' use of free periods through personalized academic activities. It serves as a reference for developers, stakeholders, and project managers to ensure the system meets functional, technical, and user needs. The document helps clarify objectives, scope, and performance expectations, providing a baseline for design, development, and testing.

1.2 Scope of this Document

The application will provide an integrated platform for:

- Automating attendance recording via QR code, Bluetooth/Wi-Fi proximity, or facial recognition.
- Displaying real-time attendance on classroom displays.
- Suggesting personalized academic tasks during idle periods based on students' interests and goals.
- Generating a daily routine combining classes, free time, and long-term objectives.

The system will reduce administrative workload, improve time management, and support students' personal development. Development is expected to require 6–8 months, with moderate budget allocation for software development, deployment, and training. The app will run on mobile devices (iOS and Android) and classroom displays, requiring minimal additional infrastructure.

1.3 Overview

The Smart Curriculum Activity & Attendance App is a multi-user platform for students, teachers, and administrators. Teachers can track attendance automatically, students receive structured guidance during free time, and administrators gain insights into engagement patterns. The app supports personalized learning, efficient attendance management, and long-term planning, improving overall academic productivity.

2. General Description

2.1 Product Functions

- Automatic attendance tracking
- Real-time attendance display
- Personalized activity suggestions for students
- Daily routine planning
- Reports for teachers and administrators

2.2 User Characteristics

- **Students:** Basic familiarity with mobile apps; motivation for personal development
- **Teachers:** Moderate technical skills; interest in reducing administrative tasks
- **Administrators:** Require dashboards to monitor attendance, engagement, and productivity

2.3 Features & Benefits

- **Efficiency:** Automates attendance, reducing manual workload
- **Productivity:** Guides students in utilizing free periods effectively
- **Data-Driven Decisions:** Provides insights for better academic support
- **Personalization:** Tailors activities according to individual student profiles

2.4 User Community

- Primary users: Students and teachers
- Secondary users: College administrators, career counselors
- Stakeholders: Educational departments and policy makers

3. Functional Requirements

1. Attendance Management

- Capture attendance via QR code scanning, Bluetooth/Wi-Fi proximity, or facial recognition
- Update attendance in real-time on classroom displays and dashboards
- Generate attendance reports for individual students and classes

2. Personalized Task Assignment

- Analyze student interests, strengths, and career goals
- Suggest academic or skill-building activities during free periods
- Track completion of suggested tasks

3. Daily Routine Generation

- Combine class schedule, free periods, and personalized tasks
- Display routine in an intuitive mobile interface

4. Administrative Reports

- Generate engagement and productivity reports
- Provide analytics for decision-making and student monitoring

4. Interface Requirements

- **User Interface:** Mobile apps (iOS/Android), web dashboard for administrators
- **System Interfaces:**

- QR code scanners or camera APIs for attendance
- Bluetooth/Wi-Fi APIs for proximity detection
- Facial recognition APIs or SDKs for automated attendance
- **Data Interfaces:**
 - Database for student profiles, attendance records, and task logs
 - APIs for integration with existing institutional systems (optional)

5. Performance Requirements

- Attendance updates must occur in under 5 seconds
- Daily routine generation should be completed within 2 seconds per student
- Maximum error rate for attendance recognition: $\leq 1\%$
- System should support up to 5000 concurrent users
- Mobile app size ≤ 50 MB; low memory footprint

6. Design Constraints

- Use cross-platform mobile development frameworks (e.g., Flutter, React Native)
- Hardware limitations: minimal dependency on external devices except smartphone or classroom display
- Software constraints: secure storage for student data, GDPR/FERPA compliance
- Algorithmic constraint: facial recognition or proximity detection must respect privacy and accuracy standards

7. Non-Functional Attributes

- **Security:** Data encryption, secure authentication, privacy compliance
- **Portability:** Runs on iOS, Android, and web
- **Reliability:** 99% uptime; error handling for attendance system
- **Reusability:** Modular design for future updates and integration
- **Scalability:** Support growing number of students and multiple institutions
- **Data Integrity:** Prevents unauthorized changes to attendance records

8. Preliminary Schedule and Budget

- **Project Duration:** 6–8 months
 - Requirement Analysis & Design: 1.5 months
 - Development: 4 months
 - Testing & Deployment: 1–1.5 months
- **Estimated Budget:** ₹32,00,000 – ₹48,00,000 (INR)
 - Development: ₹20,00,000
 - Testing: ₹6,50,000
 - Deployment & Training: ₹5,50,000 – ₹7,50,000

Chapter 3: Class Modeling

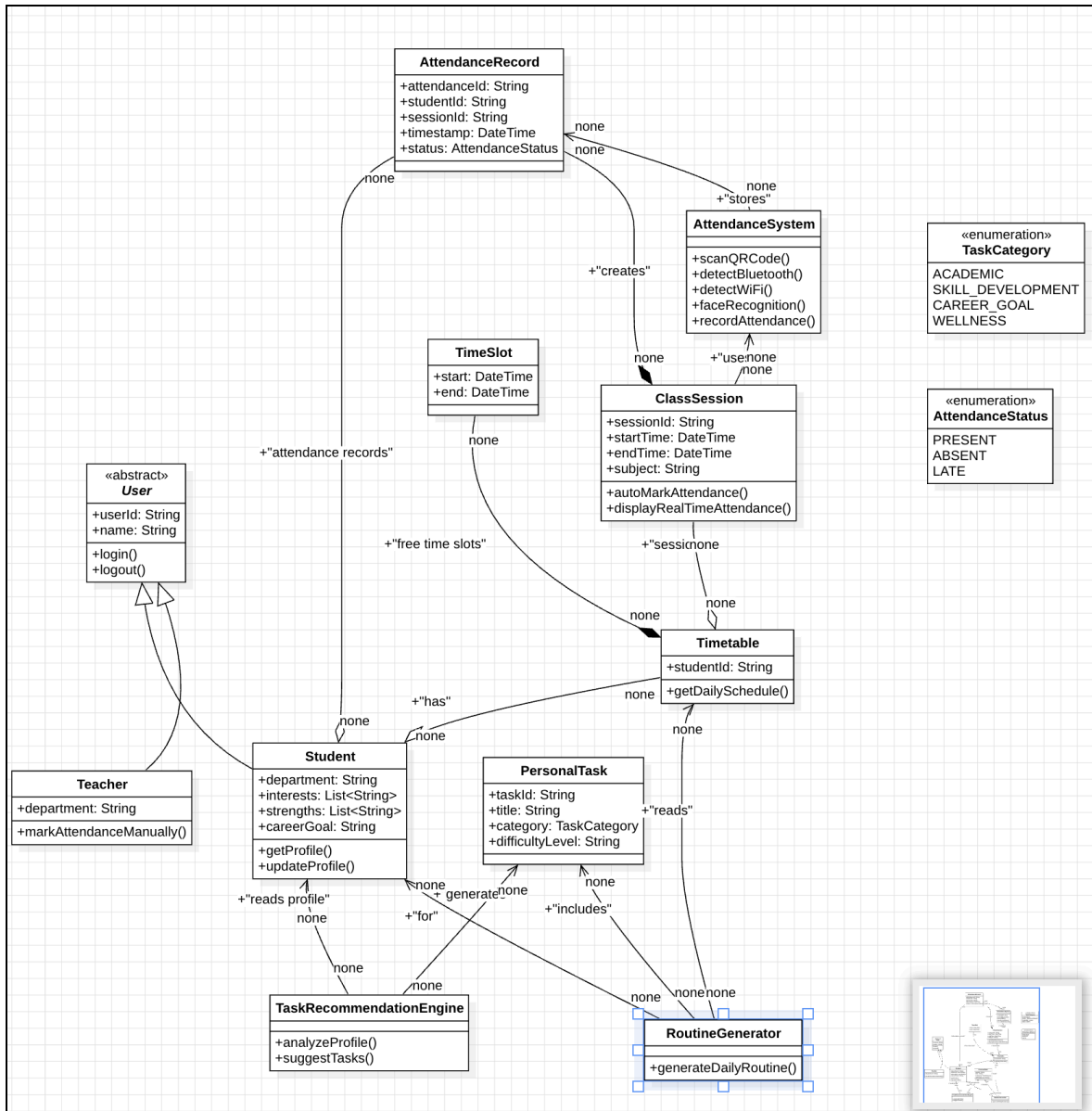


Figure 1: Class Diagram

The design is logically divided into four primary subsystems:

I. Core User Model

This model defines the base actors and their roles within the system.

User (Abstract Class): This abstract base class provides a common interface for all system users. It establishes foundational attributes (`userId`, `name`) and essential behaviors (`login()`, `logout()`). As an abstract class, it cannot be instantiated directly but serves as a parent for concrete user types.

Student: This class inherits from `User` and represents the primary consumer of the system's services. Its relevance is twofold: 1) It is the subject for whom attendance is tracked, and 2) It provides a detailed profile (including interests, strengths, and `careerGoals`) that serves as the primary input for the `TaskRecommendationEngine`.

Teacher: This class also inherits from `User` and represents an administrative actor. The teacher's primary function is to manage attendance records, with the specific ability to `markAttendanceManually()`. This function provides a necessary mechanism for manual overrides and corrections within the attendance subsystem.

II. Attendance Tracking Subsystem

This subsystem contains the core logic and data structures for capturing and storing attendance data.

AttendanceSystem: This class acts as the central controller for all attendance-related operations. It encapsulates the "advanced" logic for capturing attendance via multiple methods, including `scanQRCode()`, `detectBluetooth()`, `detectWiFi()`, and `faceRecognition()`. Its core responsibility is to identify a student's presence and subsequently `recordAttendance()`.

ClassSession: This class represents a specific, scheduled academic event (e.g., a lecture or lab). It serves as the primary context in which attendance is recorded, holding data such as `sessionId`, `subject`, and `dateTime`. It contains methods to `autoMarkAttendance()` (likely delegating to the `AttendanceSystem`) and `displayRealTimeAttendance()`.

AttendanceRecord: This class is the fundamental data entity that memorializes an attendance event. An instance of `AttendanceRecord` is created by the `AttendanceSystem` to associate a

single Student with a specific ClassSession. It stores a timestamp and an AttendanceStatus. The aggregation of these records constitutes a student's complete attendance history.

AttendanceStatus (Enumeration): This enumeration defines a constrained set of valid values (PRESENT, ABSENT, LATE) for the status attribute of an AttendanceRecord. This ensures data integrity and consistency across the system.

III. Scheduling and Task Management Subsystem

This subsystem manages the temporal components of the system, including student schedules and personal tasks.

TimeSlot: This is a simple data structure that defines a discrete block of time using start and end DateTime attributes. It serves as the basic building block for the Timetable and ClassSession.

Timetable: This class represents the complete personal schedule for a single Student. It aggregates all associated ClassSessions and TimeSlot objects. A critical function of this class is to identify and expose "free time slots," which is essential data consumed by the RoutineGenerator.

PersonalTask: This entity represents a recommended, non-academic activity intended for the student's personal or professional development. Each task includes a title, description, TaskCategory, and difficultyLevel. These tasks are the primary output of the TaskRecommendationEngine.

TaskCategory (Enumeration): This enumeration provides a controlled vocabulary for classifying PersonalTask objects (e.g., ACADEMIC, SKILL DEVELOPMENT, CAREER, GOAL, WELLNESS). This facilitates filtering, sorting, and management of recommended tasks.

IV. Task Recommendation Subsystem

This subsystem comprises the advanced, intelligent features of the application, transforming it from a simple utility into a proactive assistant.

TaskRecommendationEngine: This class functions as the core logic engine for the recommendation feature. It is a service class responsible for analyzeProfile(), which reads and interprets the Student's profile data. Based on this analysis, it suggestTasks() by instantiating and configuring a list of relevant PersonalTask objects.

RoutineGenerator: This class provides the logic to make the recommendations actionable. It synthesizes data from two other components: it retrieves the list of PersonalTask objects from

the TaskRecommendationEngine and the "free time slots" from the Student's Timetable. Its primary method, generateDailyRoutine(), intelligently allocates these tasks into the student's available free time, thus creating a holistic daily schedule.

Chapter 4: State Modeling

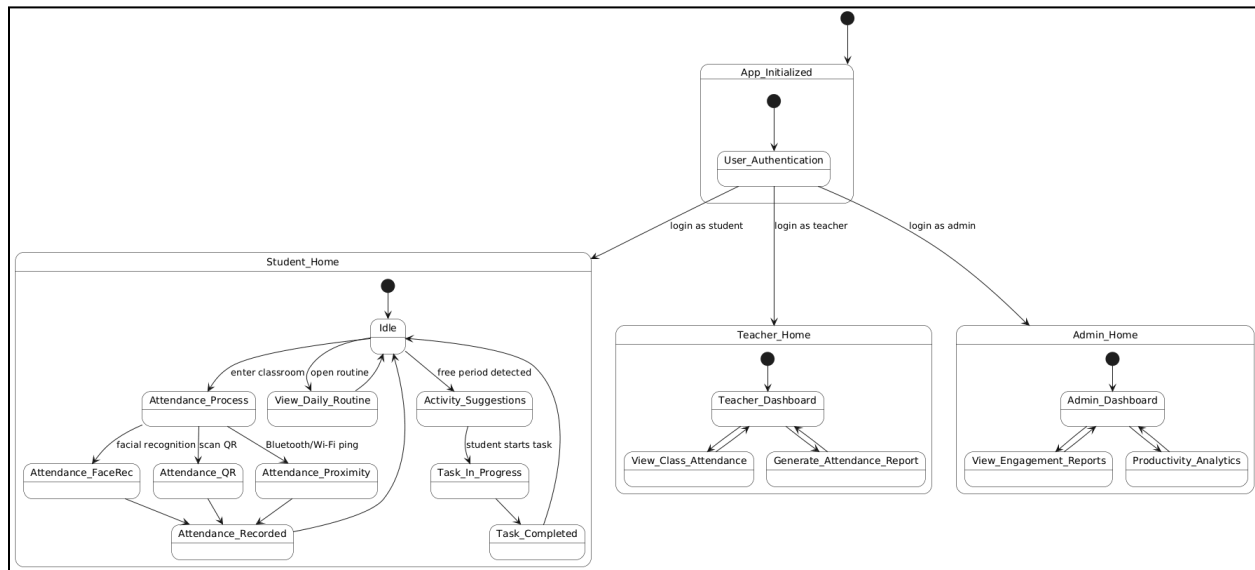


Figure 2: State Diagram

The diagram outlines the state flow of a multipurpose attendance and activity-management application used by students, teachers, and administrators. The system begins at initialization and transitions to a user authentication phase, where the user's role determines the next interface: Student Home, Teacher Home, or Admin Home. Each role leads to a distinct set of functional states tailored to the responsibilities and needs of that user group.

Within the Student Home flow, the primary states include idle activity, attendance processing, viewing daily routines, and receiving activity suggestions during free periods. Attendance can be recorded through QR scanning, proximity detection, or facial recognition, all leading to a unified attendance-recorded state before returning to idle. Students may also engage with suggested tasks, moving from task initiation to completion. Teachers, on the other hand, navigate between a dashboard, class attendance views, and attendance report generation. Administrators have access to engagement reports and productivity analytics, cycling between these functions and the admin dashboard.

Chapter 5: Interaction Modeling

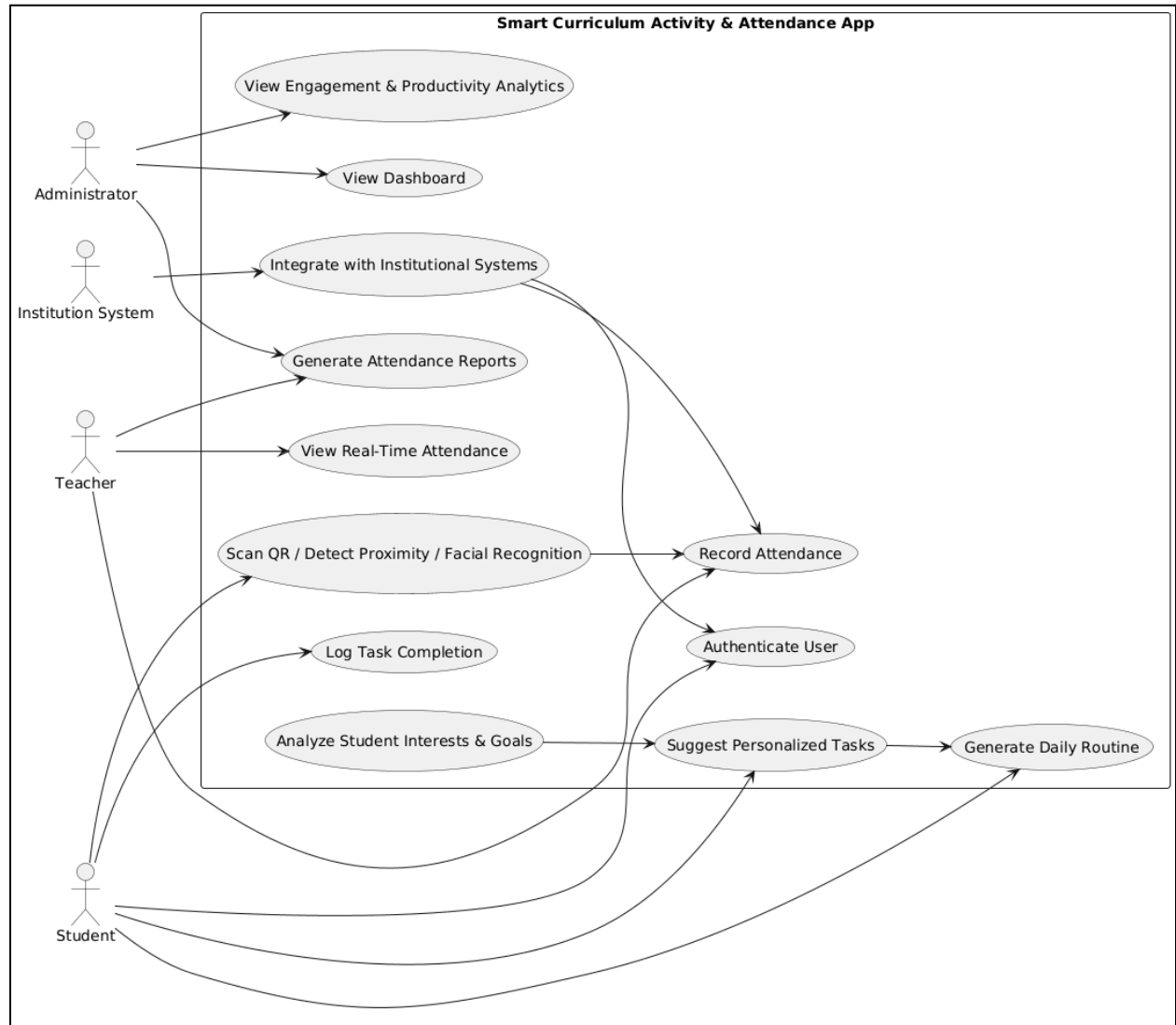


Figure 3: Use Case Diagram

Use Case Relevance:

Authenticate User

This use case is essential for verifying the identity of students, teachers, and administrators before granting access to system features. It ensures security and protects sensitive academic and attendance data.

Record Attendance

This is a core function of the application, enabling automated and efficient collection of attendance data. It reduces manual workload and minimizes errors associated with traditional attendance methods.

Scan QR / Detect Proximity / Facial Recognition

These are the operational mechanisms through which attendance is captured. The use case is relevant as it provides multiple options for institutions with varying infrastructure and privacy requirements.

View Real-Time Attendance

This allows teachers to monitor which students are present in class at any given moment. The use case enhances transparency and classroom management efficiency.

Generate Attendance Reports

This use case supports teachers and administrators by offering structured insights into attendance trends, absenteeism, and student participation. It plays a role in academic monitoring and compliance reporting.

Analyze Student Interests and Goals

This use case allows the system to understand individual student profiles. It is relevant for enabling personalized learning pathways and meaningful task recommendations.

Suggest Personalized Tasks

This expands the role of the system beyond attendance tracking into academic support and productivity improvement. It guides students to make better use of free periods.

Log Task Completion

This allows the system to track a student's engagement with recommended activities. It helps improve future recommendations and enables progress monitoring.

Generate Daily Routine

This use case combines timetable data, task suggestions, and free periods to deliver a structured daily plan for students. It is relevant for helping students manage time effectively.

View Dashboard

This allows administrators to access summarized institutional data. It supports high-level decision-making and oversight.

View Engagement and Productivity Analytics

This provides deeper insights into how students are utilizing their time and interacting with recommended tasks. The use case helps administrators assess the effectiveness of academic programs.

Integrate with Institutional Systems

This enables the app to work with existing campus platforms for user data, schedules, or attendance records. It ensures consistency across systems and reduces duplicate administrative effort.

Actor Relevance:

Student

The student is the primary beneficiary, using the app for attendance, personalized task suggestions, and daily planning. Their interactions drive the system's learning and recommendation processes.

Teacher

The teacher relies on the system for efficient attendance management and classroom monitoring. Their role reduces administrative tasks and improves classroom accuracy.

Administrator

The administrator focuses on institutional oversight, using dashboards and analytics to monitor attendance trends, engagement levels, and productivity. They support planning and policy decisions based on collected data.

Institution System

This represents external academic platforms used by the institution. Its relevance lies in the need for synchronization of student information, class schedules, and attendance data to maintain consistency and reduce manual work.

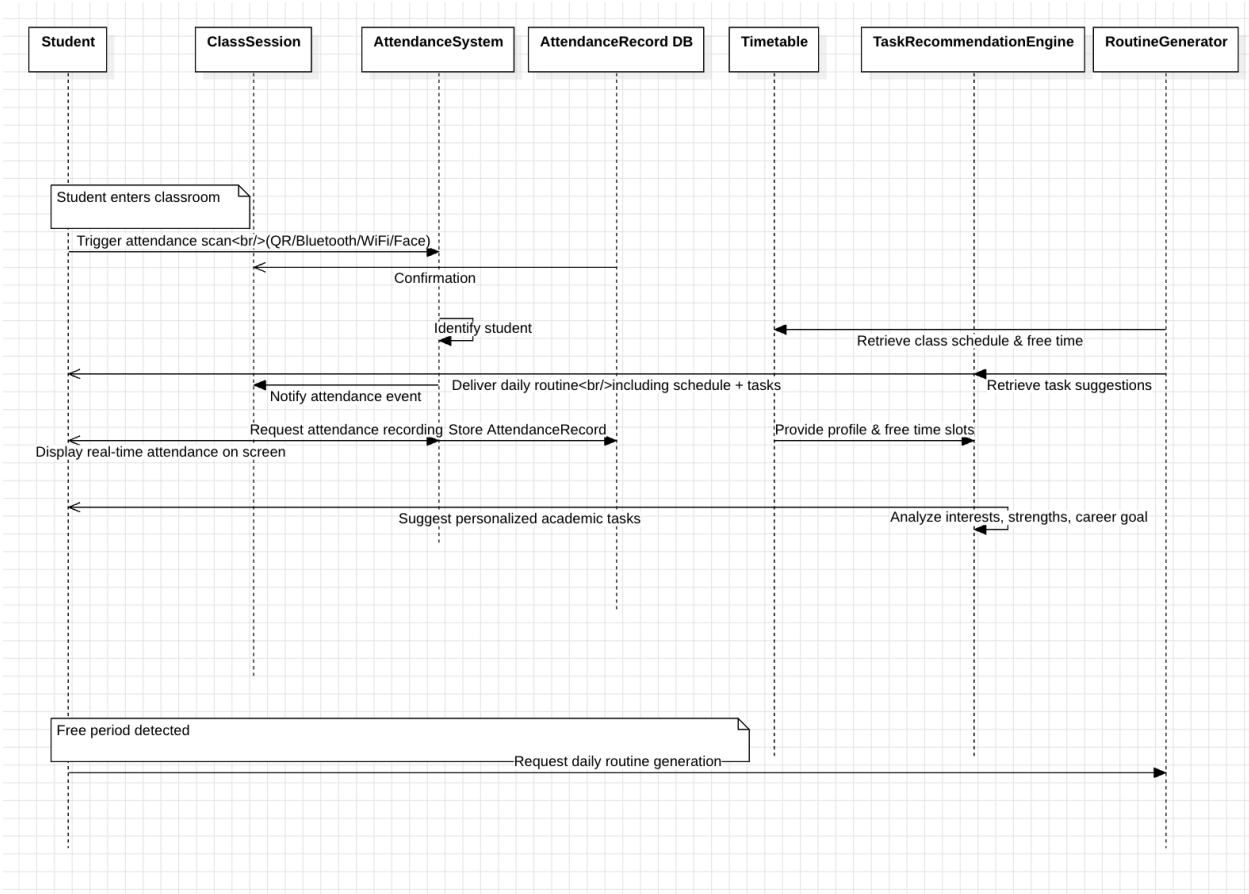


Figure 4: Sequence Diagram

This sequence diagram illustrates the dynamic interactions between the various objects within the Automated Attendance and Task Recommendation System. It details the

chronological flow of messages required to perform two key functionalities: automated attendance marking upon student entry and the generation of a personalized daily routine.

Participating Objects (Lifelines):

The diagram involves the following key entities:

- Student: The primary actor initiating the interaction.
- ClassSession: The object representing the current academic session.
- AttendanceSystem: The controller responsible for processing attendance data.
- AttendanceRecord DB: The database entity for storing permanent attendance logs.
- Timetable: The entity containing the student's schedule and availability.
- TaskRecommendationEngine: The logic component responsible for analyzing profiles and suggesting tasks.
- RoutineGenerator: The component that synthesizes the final daily schedule.

Sequence of Events

Phase 1: Automated Attendance Tracking

The interaction begins when the Student enters the classroom. This physical event triggers the attendance scan via the AttendanceSystem using one of the supported modalities (QR code, Bluetooth, Wi-Fi, or Facial Recognition).

Upon receiving the trigger, the AttendanceSystem identifies the student and sends a "Store AttendanceRecord" message to the AttendanceRecord DB. The database processes this request and returns a "Confirmation" message to the AttendanceSystem, ensuring data persistence.

Once the record is secured, the AttendanceSystem sends a notification to the ClassSession object regarding the attendance event. Consequently, the ClassSession triggers a display action, updating the classroom screen to show the Real-time Attendance status to the Student and the class.

Phase 2: Routine Generation and Task Recommendation

Parallel to or triggered by the system's state (such as the detection of a free period), the RoutineGenerator initiates the planning process. It sends a request to the Timetable object to "Retrieve class schedule & free time," identifying available slots in the student's day.

With this temporal data, the RoutineGenerator communicates with the TaskRecommendationEngine. It passes the necessary context (student profile and free time slots) and requests task suggestions. The TaskRecommendationEngine performs an internal analysis of the student's interests, strengths, and career goals.

After the analysis, the TaskRecommendationEngine returns a list of personalized academic or developmental tasks to the RoutineGenerator. Finally, the RoutineGenerator compiles the fixed academic schedule with these recommended tasks and delivers the complete Daily Routine to the Student.

The sequence diagram effectively demonstrates the system's ability to handle immediate, hardware-triggered events (attendance) and complex, data-driven processing (recommendations) in a cohesive flow. It highlights the separation of concerns, where specific objects handle storage, logic, and user interaction independently.

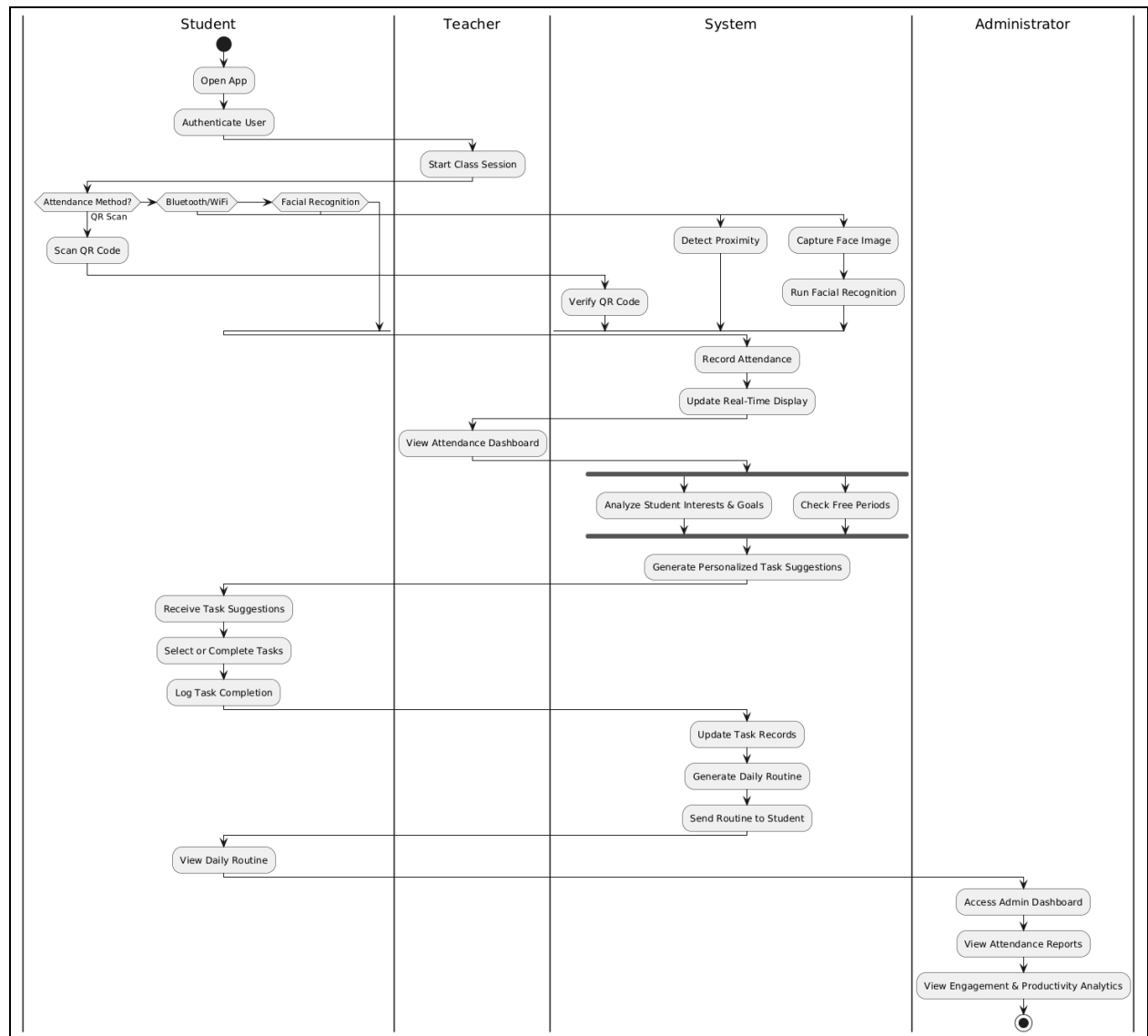


Figure 5: Activity Diagram

The activity diagram illustrates the workflow of the Smart Curriculum Activity & Attendance App, highlighting the interactions among four primary actors: the student, the teacher, the system, and the administrator. The process begins with the student launching the application and authenticating their identity. Once authenticated, the teacher initiates the class session, prompting the attendance process.

The attendance workflow includes three possible methods: QR code scanning, Bluetooth or Wi-Fi proximity detection, and facial recognition. Based on the student's chosen method, the system verifies the QR code, detects proximity, or processes the facial recognition data. After successful verification, the system records the attendance and updates the real-time

classroom display. The teacher can then view the attendance dashboard to monitor student participation instantly.

The system simultaneously carries out two analyses: assessing student interests, strengths, and goals, and identifying available free periods. Using the results of these analyses, the system generates personalized academic or skill-building tasks for the student. The student receives the suggested tasks, chooses or completes them, and logs their progress. The system updates these task records to maintain an accurate profile of student engagement.

Following this, the system generates a complete daily routine that integrates the student's class schedule, free periods, and personalized tasks. This routine is delivered to the student, who can then view their planned day within the app.

Lastly, administrators can access their dashboard to review attendance reports, engagement metrics, and productivity analytics. This supports informed decision-making and institutional monitoring. The diagram as a whole captures the end-to-end flow of attendance automation, personalized learning support, and administrative oversight.

Chapter 6: UI Design with Screenshots

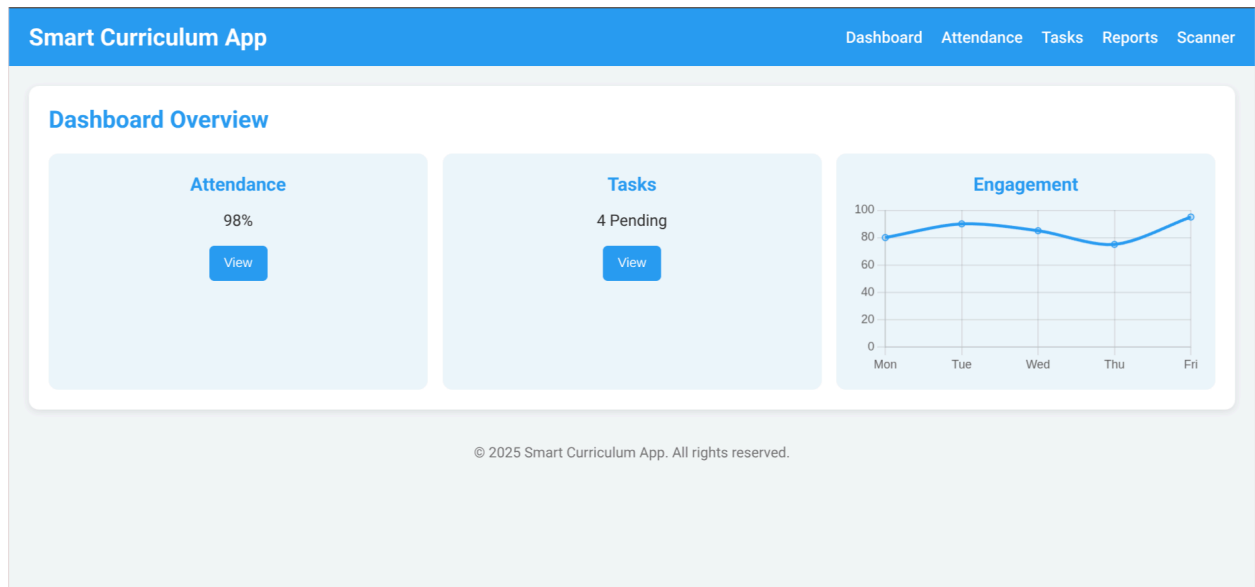


Image 1: Dashboard Overview

Figure above shows the dashboard for the application. The overview consists of Attendance, Tasks and Engagement sections. Each section is a separate page itself.

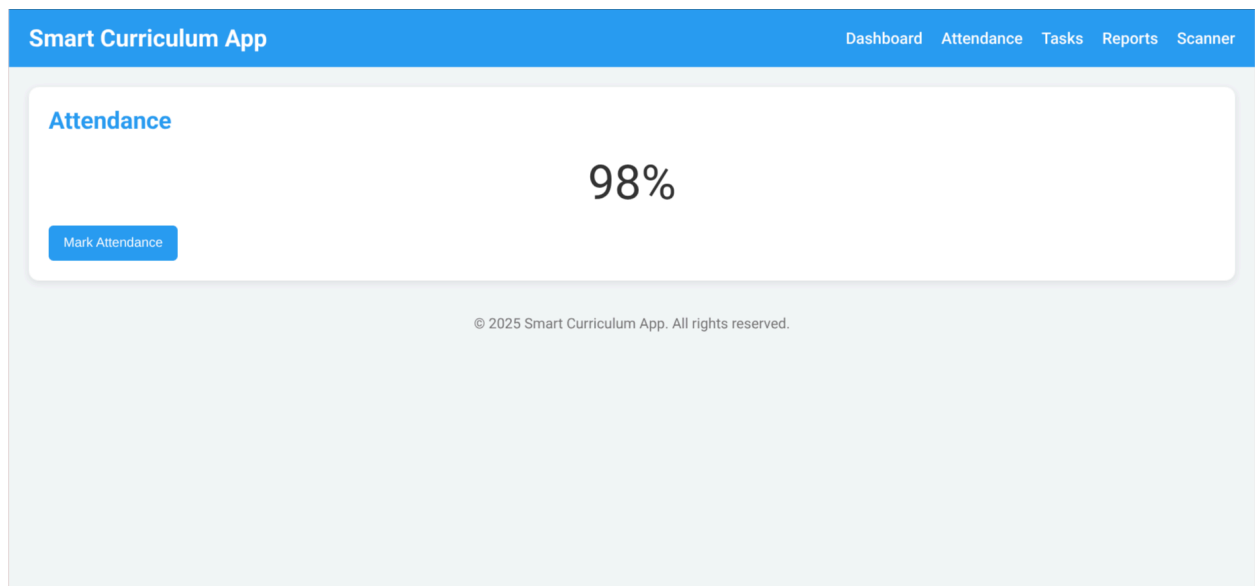


Image 2: Attendance Overview

Figure above shows the attendance section for the teacher to input the attendance of a given student. By clicking on the “Mark Attendance” button, the teacher increments the attendance value by one.

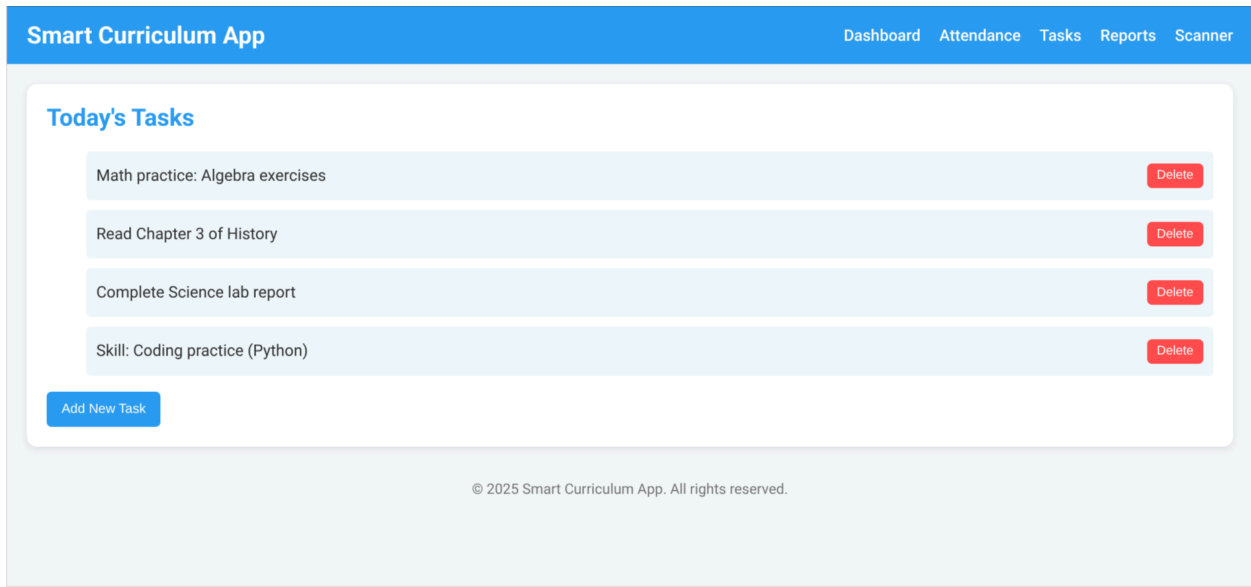


Image 3: Task Overview

The Figure above shows the Tasks section for the student. A student can add a task using the “Add New Task” button, and on completion of the task, can delete them individually using the Delete button.

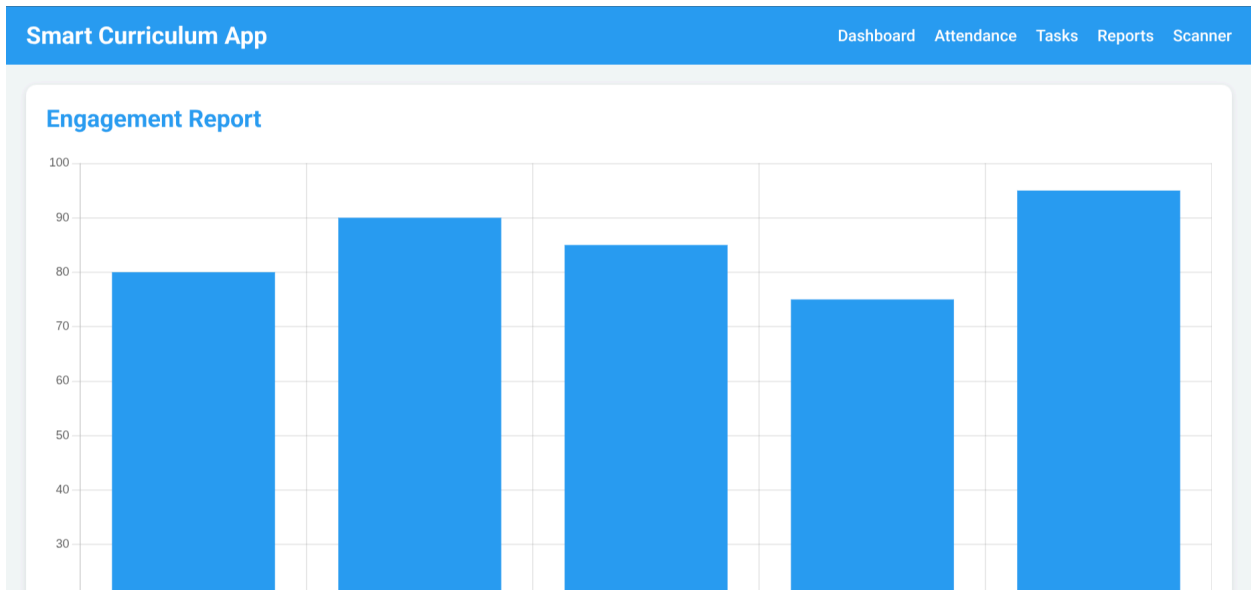


Image 4: Engagement Report Screen

The Figure above shows the Engagement report for the given application. This shows the engagement of students in the teacher's class across the week as a percentage of total satisfaction.

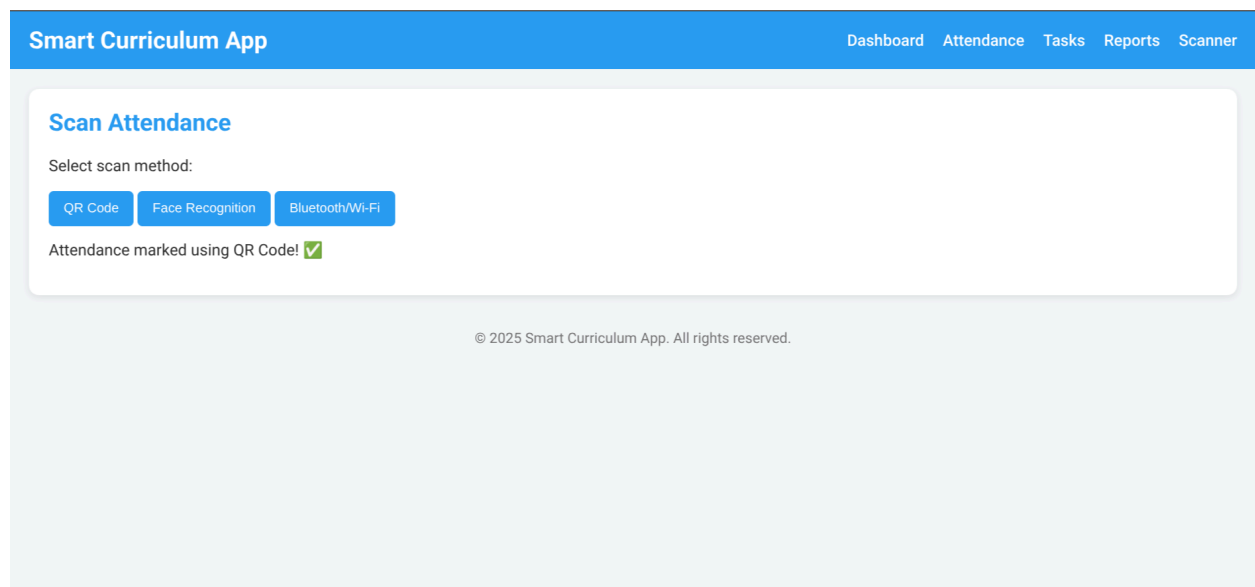


Image 5: Attendance Upload Screen

The Figure above shows the Scanner. Students can mark attendance using either QR Code, Face Recognition or Wi-Fi, to reduce students from marking themselves present, even though they are not.